UNIVERSIDAD DE SAN CARLOS DE GUATEMALA

Facultad de Ingeniería

Escuela de Ingeniería en Ciencias y Sistemas

Laboratorio de Introducción a la Programación y Computación 1

TAREA 4

Sección de investigación

Programación Orientada a Objetos y Java Swing

Estudiante: Jimena Sofia Ismalej Fernandez

Carné: 202500375

Introducción

En esta investigación se presentan los conceptos que se necesitan para desarrollar una aplicación en Java que permita registrar cursos, agregar tareas académicas y convertir temperaturas. Los temas se relacionan con la Programación Orientada a Objetos (POO), la creación de interfaces con Swing, el manejo de eventos y la validación de datos.

1. Programación Orientada a Objetos: clase, objeto, atributo y método

La Programación Orientada a Objetos (POO) es una forma de organizar un programa utilizando clases y objetos. Ayuda a separar las responsabilidades de cada parte del sistema para que el código sea más fácil de comprender, mantener y reutilizar.

Clase: es un modelo que define qué información guarda un objeto y qué acciones puede realizar. Por ejemplo, la clase Curso describe los datos de un curso académico.

Objeto: es una instancia creada a partir de una clase. Un objeto puede representar el curso IPC1, con su código y tutor.

Atributo: es una variable que almacena una característica del objeto, como codigo, nombre o tutor.

Método: es una acción o función definida dentro de una clase, como agregarTarea() o getNombre().

```
Curso curso = new Curso("0770", "IPC1", "Tutor");
String nombreCurso = curso.getNombre();
```

En el ejemplo se crea un objeto de tipo Curso y después se consulta su nombre mediante un método.

2. Los cuatro pilares de la POO

Encapsulamiento: consiste en proteger los datos de un objeto y controlar cómo se consultan o modifican. En Java se utiliza, por ejemplo, private para los atributos y métodos públicos como los getters y setters.

```
private String nombre;
public String getNombre() { return nombre; }
```

Herencia: permite crear una clase a partir de otra para reutilizar sus características. En la aplicación, VentanaPrincipal extiende JFrame para aprovechar el funcionamiento de una ventana.

```
public class VentanaPrincipal extends JFrame {
    // Componentes y eventos de la ventana
}
```

Polimorfismo: permite que objetos de distintas clases respondan de diferentes maneras ante una misma operación o método. Por ejemplo, cada clase puede adaptar el método toString() para mostrar su información.

Abstracción: consiste en representar las características importantes de un elemento sin mostrar todos los detalles de su funcionamiento. Así, se puede trabajar con un objeto Curso sin conocer cómo está organizada internamente toda su información.

Estos principios favorecen un código más ordenado. No es necesario forzar el uso de los cuatro en todas las clases; lo importante es comprender su utilidad y aplicarlos cuando corresponda.

3. Diseño de clases para cursos y tareas académicas

Diseñar las clases antes de programar ayuda a decidir qué datos y métodos tendrá cada objeto. Para el gestor académico de la tarea se pueden usar dos clases principales: Curso y TareaAcademica.

Clase	Atributos	Métodos o responsabilidades
Curso	codigo, nombre, tutor, tareas	Agregar tareas y mostrar la información del curso.
TareaAcademica	titulo, descripcion, fechaEntrega	Guardar y consultar los datos de una tarea.

La relación entre las dos clases es que un curso puede tener varias tareas. Para guardar esta asociación, la clase Curso puede tener un ArrayList<TareaAcademica>. Así, cada tarea queda registrada dentro del curso que le corresponde.

```
private ArrayList<TareaAcademica> tareas = new ArrayList<>();
public void agregarTarea(TareaAcademica tarea) {
    tareas.add(tarea);
}
```

La clase VentanaPrincipal se encarga de mostrar y recibir información en la pantalla, mientras que las clases Curso y TareaAcademica representan los datos. Esta separación evita colocar toda la lógica en una sola clase.

4. Java Swing y sus componentes principales

Java Swing es una biblioteca que sirve para construir interfaces gráficas de escritorio. Permite agregar ventanas, textos, campos de entrada y botones mediante código Java. Los componentes utilizados en esta tarea son los siguientes:

JFrame: Ventana principal de la aplicación. Permite colocar los demás componentes.

JPanel: Contenedor que ayuda a agrupar y organizar botones, etiquetas y campos.

JLabel: Muestra textos de guía o resultados, como “Nombre del curso”.

JTextField: Permite ingresar texto en una sola línea, por ejemplo un código o una temperatura.

JButton: Botón que el usuario presiona para ejecutar una acción.

JTextArea: Muestra varias líneas de texto, como el listado de cursos y tareas.

JOptionPane: Muestra cuadros de diálogo para informar de un registro exitoso o señalar un error.

```
JButton boton = new JButton("Registrar curso");
JTextField campo = new JTextField();
JOptionPane.showMessageDialog(null, "Curso registrado");
```

También se pueden utilizar JComboBox para seleccionar el curso y JTabbedPane para dividir la aplicación en pestañas. La guía pide crear la interfaz manualmente, sin utilizar el diseñador de arrastrar y soltar.

5. Manejo de eventos en Java: ActionListener

Un evento es algo que sucede cuando el usuario interactúa con la interfaz, por ejemplo hacer clic en un botón. ActionListener se utiliza para indicar qué instrucciones deben ejecutarse cuando ocurre una acción.

En Swing se registra un listener mediante addActionListener(). Puede escribirse con una clase que implemente ActionListener o mediante una expresión lambda, como se hace en la aplicación de la tarea.

```
JButton btnRegistrar = new JButton("Registrar curso");
btnRegistrar.addActionListener(e -> registrarCurso());
```

En este ejemplo, al presionar el botón se llama al método registrarCurso(). Dentro de ese método se revisan los campos y se guarda el curso si los datos son válidos. De forma similar, los botones de conversión llaman al método que calcula Fahrenheit o Kelvin.

6. Validación de entradas numéricas y textuales en Java

La validación permite comprobar los datos antes de utilizarlos. Esto evita que se registren cursos incompletos o que el programa falle cuando se escribe un valor incorrecto en un campo numérico.

Validar texto: se puede aplicar trim() para quitar espacios al inicio y final e isEmpty() para revisar si un campo quedó vacío.

```
String nombre = txtNombre.getText().trim();
if (nombre.isEmpty()) {
    JOptionPane.showMessageDialog(this, "Ingrese el nombre");
    return;
}
```

Validar números: Double.parseDouble() convierte un texto a un número decimal. Si el texto no es válido, genera NumberFormatException, que se puede controlar con try-catch.

```
try {
    double celsius = Double.parseDouble(txtCelsius.getText());
} catch (NumberFormatException ex) {
    JOptionPane.showMessageDialog(this, "Ingrese un numero valido");
}
```

Validar fechas: se puede comprobar que la fecha cumpla el formato solicitado (DD/MM/AAAA) y sea una fecha real mediante LocalDate y DateTimeFormatter.

Mostrar errores: con JOptionPane.showMessageDialog() se informa al usuario qué debe corregir, sin cerrar inesperadamente la aplicación.

También conviene evitar códigos de curso duplicados y comprobar que exista un curso antes de asociarle una tarea.

Conclusiones

La POO permite organizar los datos en clases como Curso y TareaAcademica, de manera que el programa sea más fácil de entender y modificar.

Java Swing facilita la creación de una interfaz en la que el usuario registra información y ejecuta acciones por medio de botones.

Las validaciones y el manejo de eventos son necesarios para que la aplicación responda correctamente y muestre mensajes claros cuando se ingresan datos incorrectos.

Referencias consultadas

Oracle. (s. f.). Object-Oriented Programming Concepts. Java Tutorials. https://docs.oracle.com/javase/tutorial/java/concepts/

Oracle. (s. f.). Interfaces and Inheritance. Java Tutorials. https://docs.oracle.com/javase/tutorial/java/IandI/

Oracle. (s. f.). Using Swing Components. Java Tutorials. https://docs.oracle.com/javase/tutorial/uiswing/components/

Oracle. (s. f.). How to Write an Action Listener. Java Tutorials. https://docs.oracle.com/javase/tutorial/uiswing/events/actionlistener.html

Oracle. (s. f.). JOptionPane (Java API Documentation). https://docs.oracle.com/en/java/javase/26/docs/api/java.desktop/javax/swing/JOptionPane.html

Anexo. Evidencias de ejecución

La siguiente captura muestra la ventana principal del gestor académico desarrollada con Java Swing y ejecutada desde NetBeans. Se observan las pestañas para cursos, tareas y conversiones de temperatura.

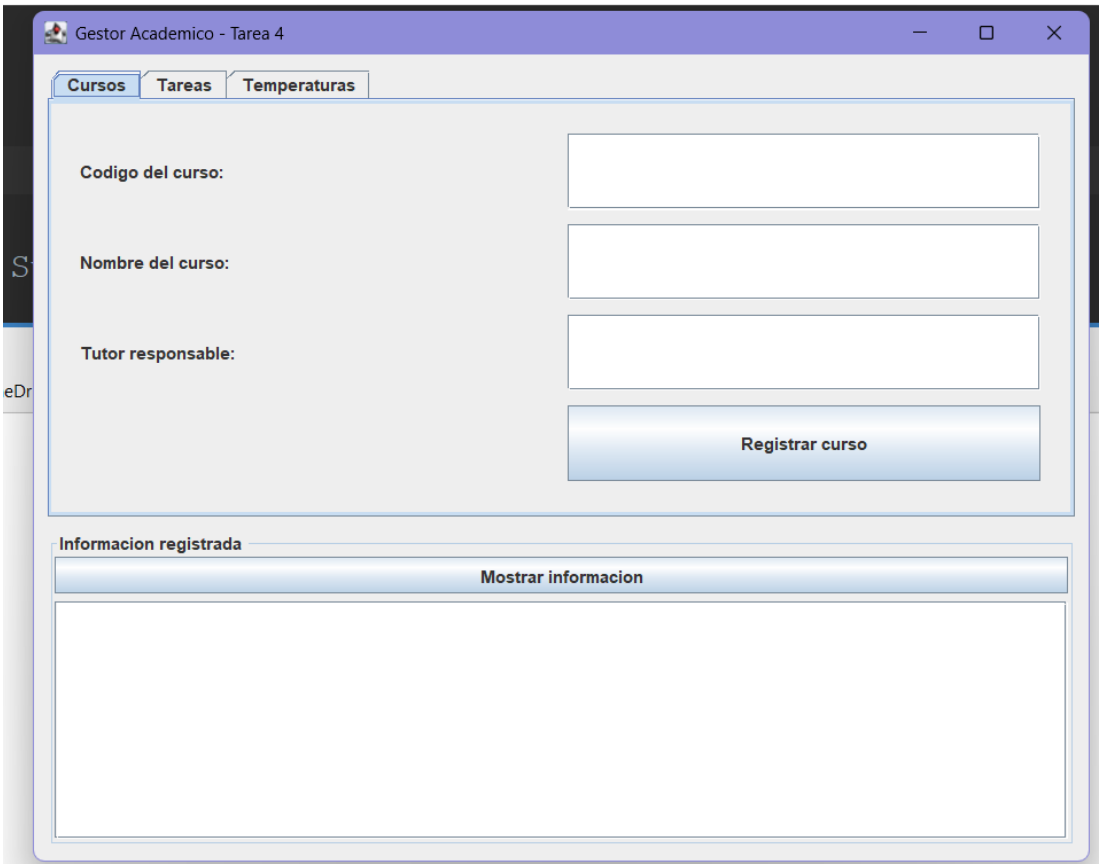


Figura 1. Pantalla principal de la aplicación Tarea 4 en ejecución.

A continuación se presentan las capturas de las pruebas realizadas: registro de cursos, selección de cursos, registro de tareas y conversión de temperaturas.

Evidencia: Registro de cursos

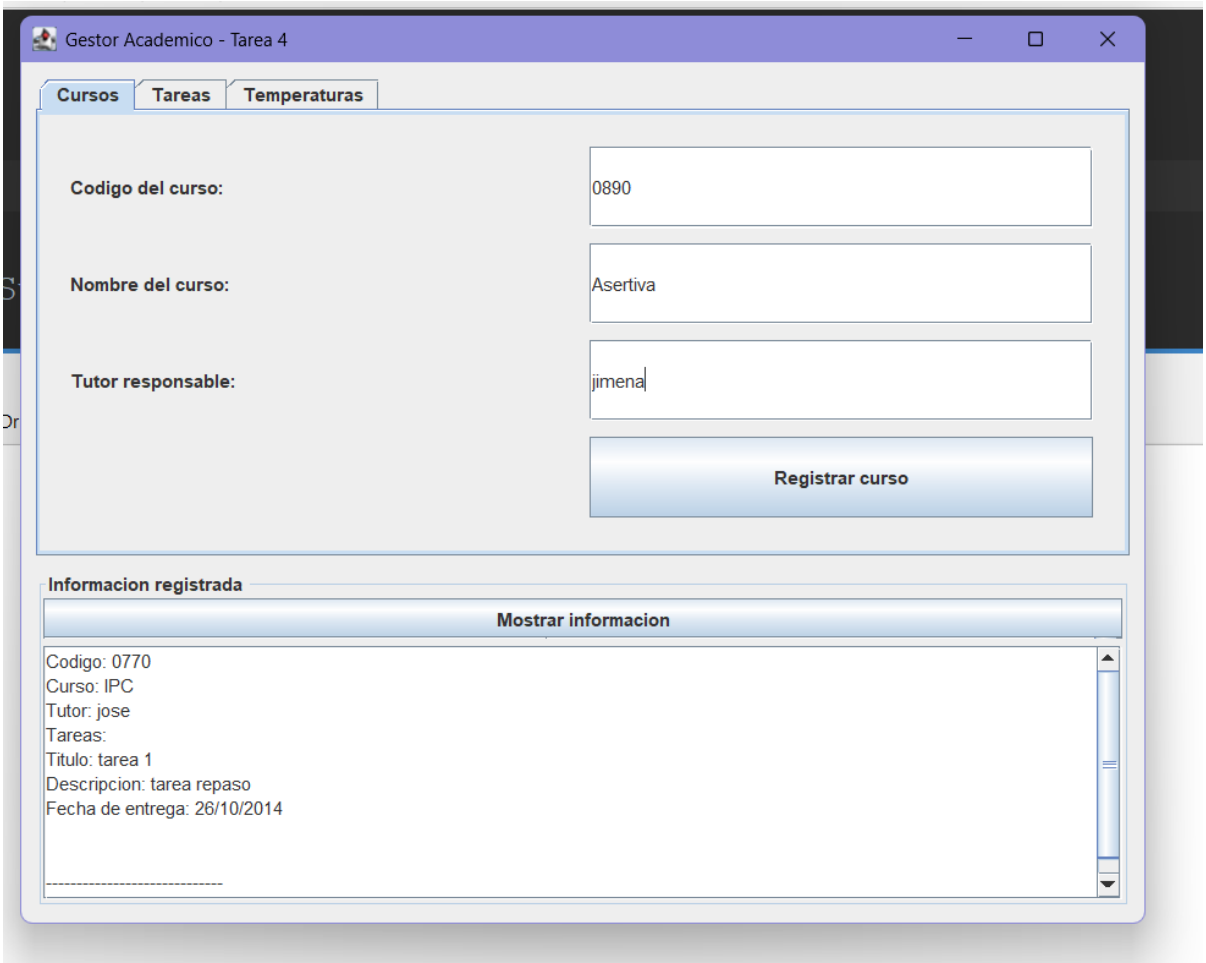


Figura 2. Ingreso de los datos del curso 0890, Asertiva y tutor responsable.

Evidencia: Registro de cursos

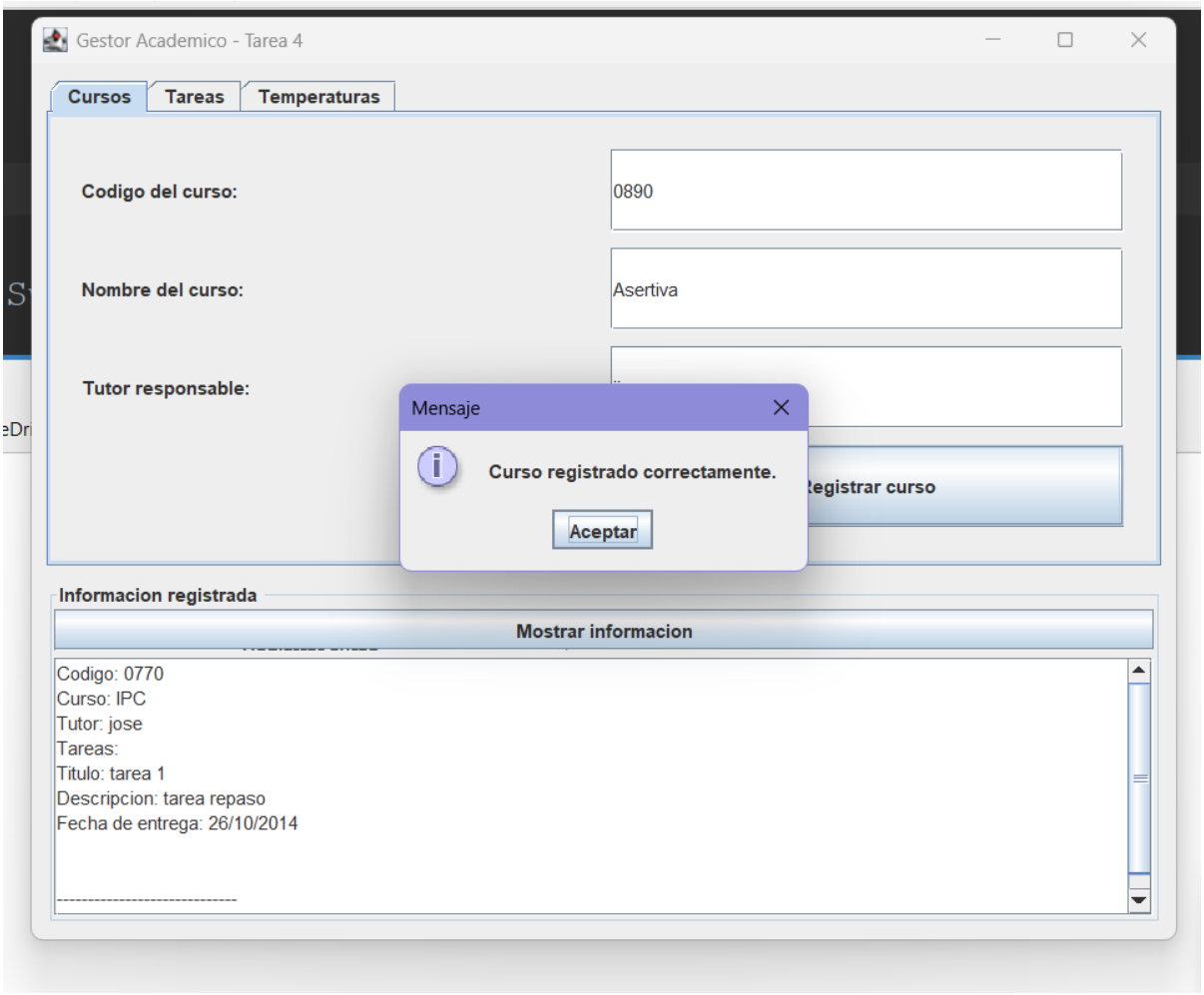


Figura 3. Mensaje que confirma el registro correcto del curso.

Evidencia: Gestión de tareas

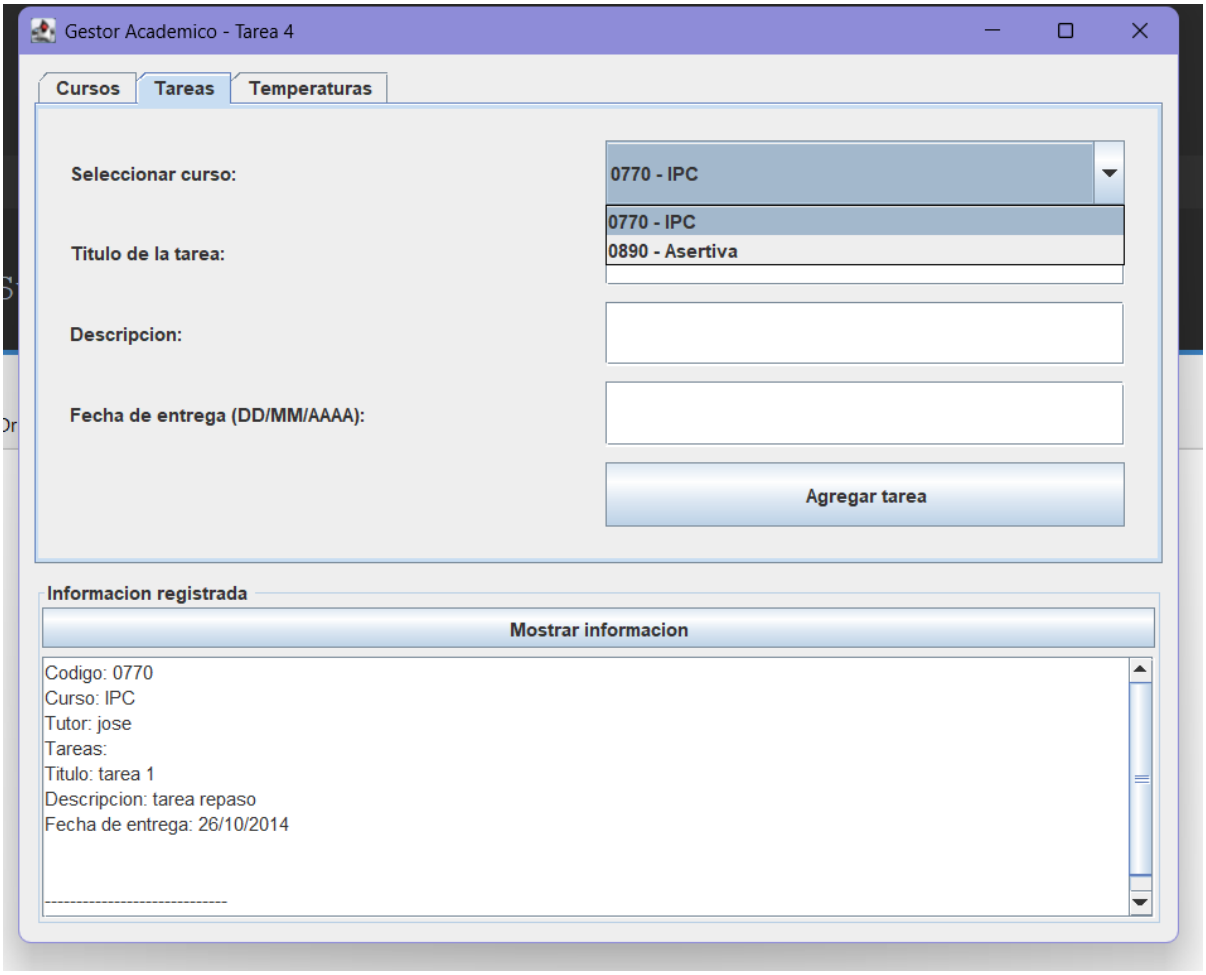


Figura 4. Lista desplegable que permite elegir uno de los cursos registrados.

Evidencia: Gestión de tareas

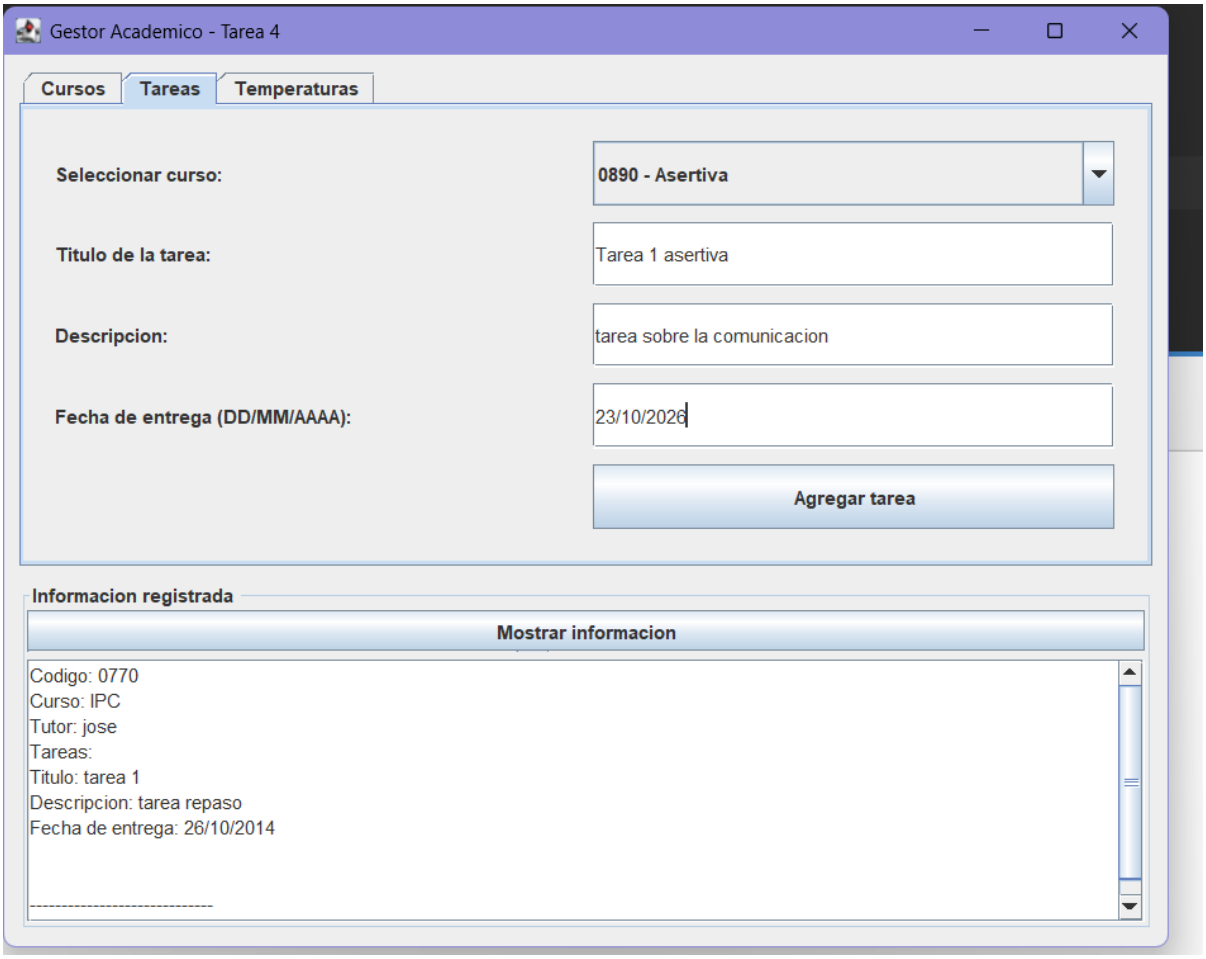


Figura 5. Registro de una tarea para el curso Asertiva, con descripción y fecha.

Evidencia: Gestión de tareas

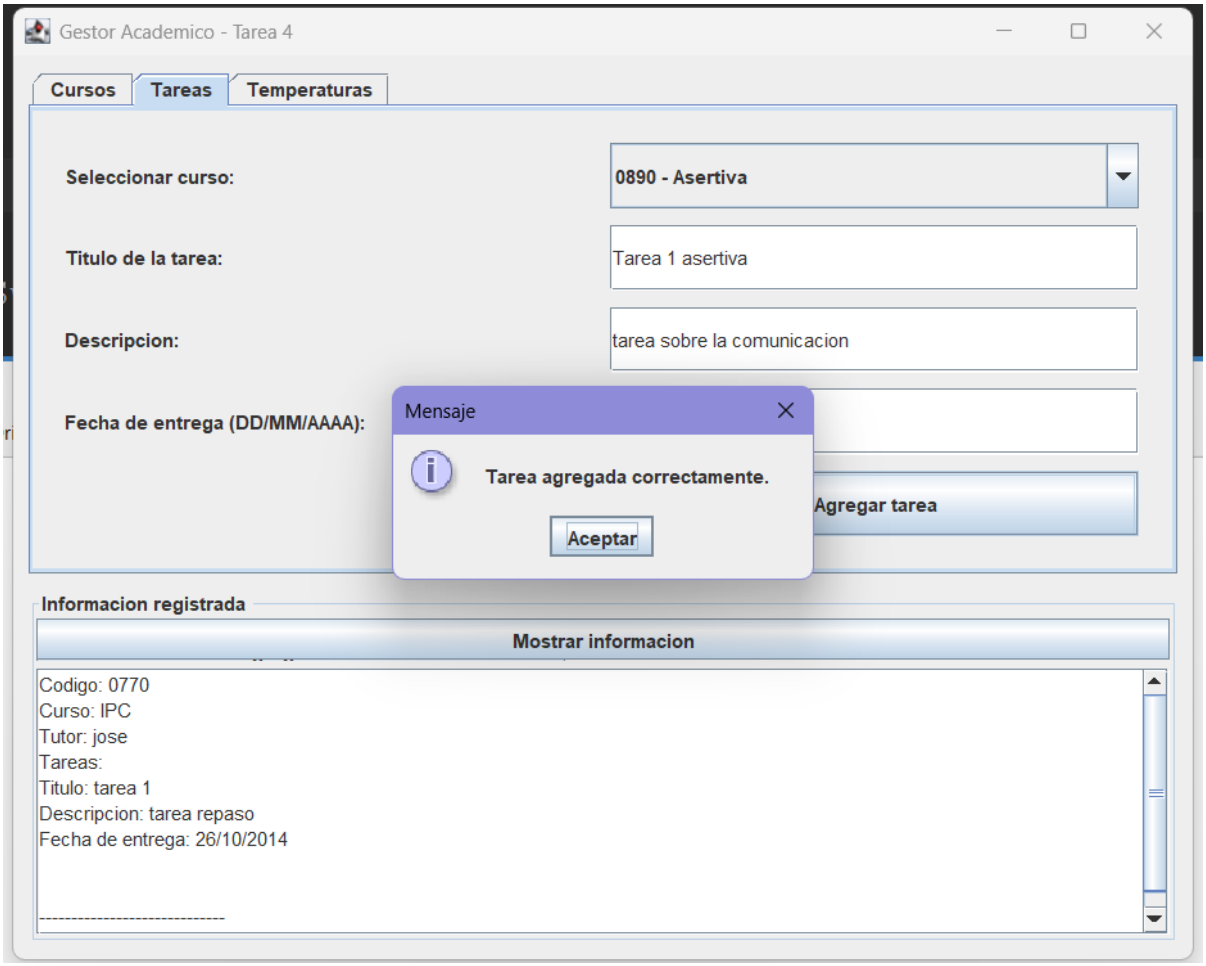


Figura 6. Mensaje que confirma que la tarea fue agregada correctamente.

Evidencia: Conversión de temperaturas

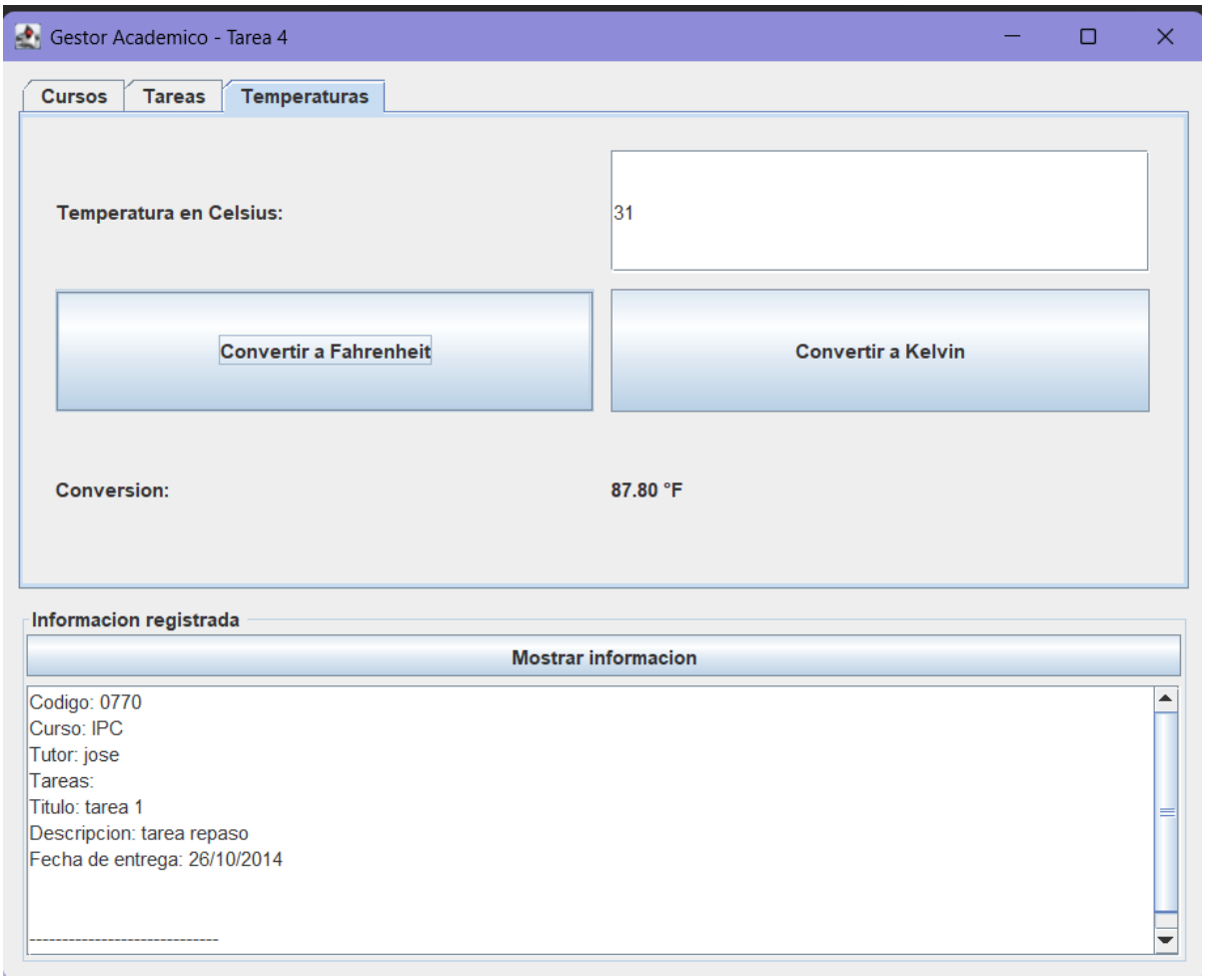


Figura 7. Conversión de 31 °C a 87.80 °F mediante el botón Fahrenheit.

Evidencia: Conversión de temperaturas

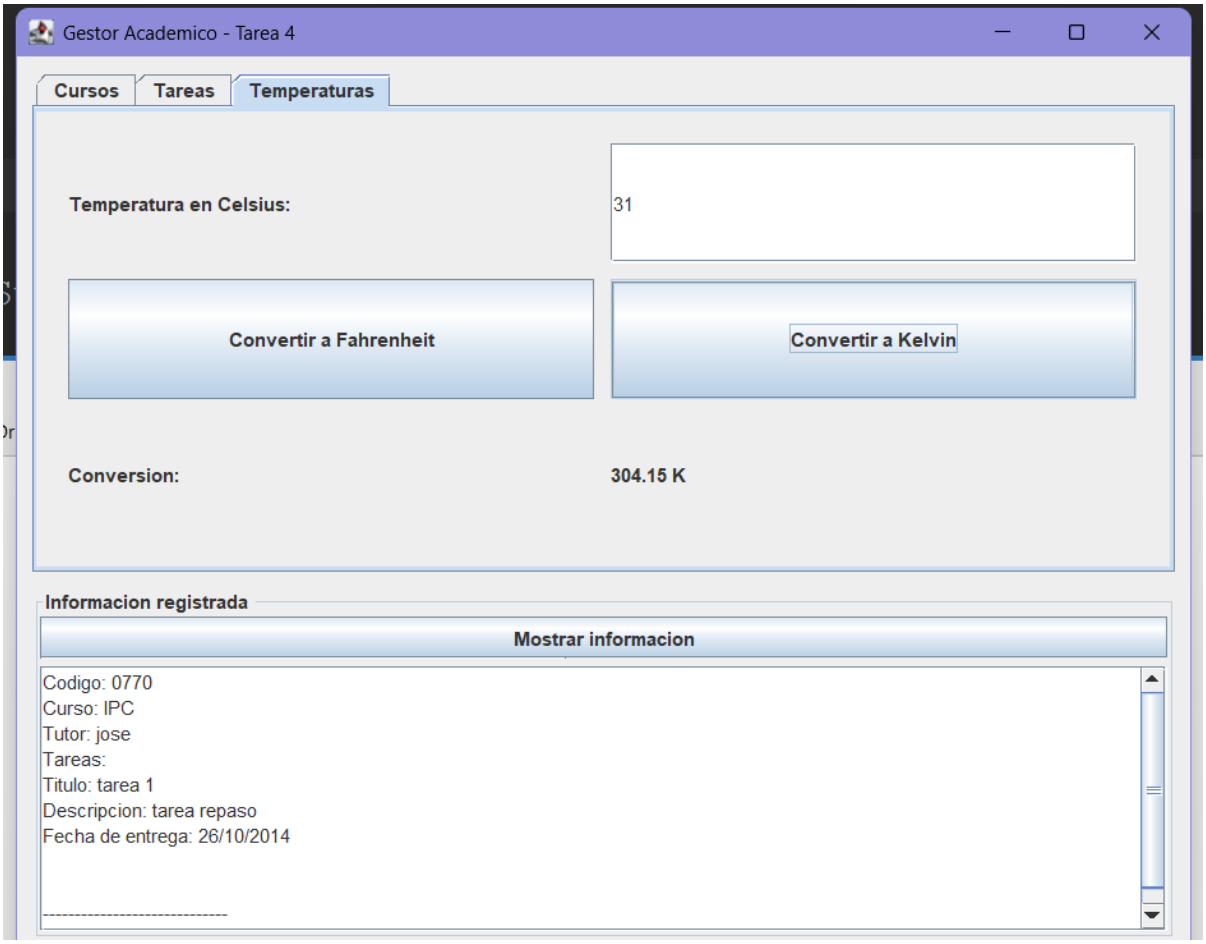


Figura 8. Conversión de 31 °C a 304.15 K mediante el botón Kelvin.

Nota: Las capturas documentan las pruebas de registro, asociación de tareas y conversión. No se adjuntó una captura específica de un mensaje de error por validación.